Flutter Theme System

Professional Light / Dark Mode Implementation

by huthaifa Ma'in [linkedIn]

Clean Architecture · ThemeExtension · Cubit · SharedPreferences

Introduction

This guide provides a complete, production-ready blueprint for implementing Light/Dark theme switching in a Flutter app. It covers how to define colors that respond to the active theme, how to manage theme state with a Cubit, how to persist the user's choice across app restarts, and the exact rules for using theme-aware colors safely inside widgets.

The system is built on three principles: every color is named by its UI role (not its hex value), a single class is the entry point for all styling, and theme state is a Cubit so it integrates naturally with the rest of your BLoC architecture.

> 🏗️ **ANALOGY:** Think of the system as a wardrobe with two complete outfit sets — one for day, one for night. AppColorsExtension is the wardrobe that holds both sets. AppStyles is the stylist who knows how to pick and combine pieces. ThemeCubit is the light switch that activates one set or the other. main.dart is the bedroom that checks what the light was set to when you last left.

Architecture Overview

The four files that make up this system and their single responsibilities:

```
┌──────────────────────────────────────────────────────────────────────────┐
│ main.dart
│ • Reads saved ThemeMode from SharedPreferences before first frame
│ • Creates ThemeCubit with the correct initial state
│ • Provides ThemeCubit to the entire widget tree via BlocProvider
│ • Connects active ThemeMode to MaterialApp
└──────────────────────────────────────────────────────────────────────────┘
                  │ emits ThemeMode
                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ ThemeCubit   (Cubit<ThemeMode>)
│ • Manages one value: ThemeMode.light | ThemeMode.dark
│ • Writes the new choice to SharedPreferences (persistence)
│ • Emits the new ThemeMode (triggers UI rebuild)
└──────────────────────────────────────────────────────────────────────────┘
                  │ activates color set
                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ AppColorsExtension   (ThemeExtension<AppColorsExtension>)
│ • Defines two complete, typed color sets: .light and .dark
│ • Each field is a semantic role: foreground, background, etc.
│ • Attached to ThemeData — Flutter activates the correct set
└──────────────────────────────────────────────────────────────────────────┘
                  │ consumed via AppStyles
                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ AppStyles
│ • Single import for every widget that needs colors or text styles
│ • colors(context)  → retrieves active AppColorsExtension
│ • Static colors    → brand colors that never change
│ • Dynamic getters  → pre-composed color + opacity utilities
│ • Text style helpers  → require context; auto-apply theme color
└──────────────────────────────────────────────────────────────────────────┘
```

File Structure

```
lib/
├── core/
│   └── theming/
│       ├── app_colors_extension.dart   ← Step 1
│       ├── app_styles.dart             ← Step 2
│       └── theme_cubit.dart            ← Step 3
└── main.dart                           ← Step 4
```

Step 1	app_colors_extension.dart — Dynamic Color Definitions

This file is the single source of truth for every color that must change between Light and Dark mode. It extends Flutter's ThemeExtension system, which allows you to attach custom typed data directly onto a ThemeData object and retrieve it anywhere in the widget tree.

🤔 **WHY?** ThemeExtension integrates into Flutter's native theme lifecycle. This means your colors participate in animated theme transitions automatically through the lerp method, and are scoped to the theme tree rather than living in a separate global state.

Core Design Rule — Name Colors by Role, Never by Value

Every field in AppColorsExtension represents a UI responsibility, not a hex value. This is the most important rule in the entire system.

```
// ❌ WRONG – named after the value
final Color darkNavyBlue;
final Color offWhite;

// ✅ CORRECT – named after the role
final Color background;   // Could be white in light, navy in dark
final Color foreground;   // Could be dark in light, pale in dark
final Color containerTop; // Top gradient stop of any card
final Color borderColor;  // Any border or divider in the app
```

📐 **RULE:** If you later need to rebrand — change the dark background from navy to dark green — you change one hex value in the dark color set. Zero widgets need to be updated because they all reference 'background', not the color itself.

What Color Fields to Define

Define one field for each distinct UI role in your app. The table below shows a recommended starting set. Add or remove fields to match your app's actual needs.

Field Name	Light Example	Dark Example	What it styles
foreground	#2D2D2D	#C0CAF5	Text, icons, labels
background	#F2F2F2	#1E2030	Scaffold / screen fill

Field Name	Light Example	Dark Example	What it styles
containerTop	#FFFFFF	#3D4070	Card top gradient stop
containerBottom	#FFFFFF	#2A2E50	Card bottom gradient stop
borderColor	#DEDEDE	#4A5580	All widget borders
appBarTop	#FFFFFF	#252840	App bar top gradient
appBarBottom	#FFFFFF	#111320	App bar bottom gradient
appBarBorder	#DCDCDC	#383950	App bar bottom border
searchBarBackground	#FFFFFF	#252840	Search field fill
searchBarBorder	#E0E0E0	#44456A	Search field border

💡 **TIP:** These are illustrative examples — replace the hex values with your actual brand palette. The field names are what matter; the values are up to your designer.

Complete File Template

```
import 'package:flutter/material.dart';

class AppColorsExtension extends ThemeExtension<AppColorsExtension> {

  // ── 1. FIELD DECLARATIONS ──────────────────────────────────────
  final Color foreground;
  final Color background;
  final Color containerTop;
  final Color containerBottom;
  final Color borderColor;
  final Color appBarTop;
  final Color appBarBottom;
  final Color appBarBorder;
  final Color searchBarBackground;
  final Color searchBarBorder;
  // Add more fields here as your UI needs grow.

  const AppColorsExtension({
    required this.foreground,
    required this.background,
    required this.containerTop,
    required this.containerBottom,
    required this.borderColor,
    required this.appBarTop,
    required this.appBarBottom,
    required this.appBarBorder,
    required this.searchBarBackground,
```

```
    required this.searchBarBorder,
  });

  // ── 2. LIGHT COLOR SET ──────────────────────────────────────────
  // Replace these values with your actual light-mode palette.
  static const AppColorsExtension light = AppColorsExtension(
    foreground:          Color(0xFF2D2D2D),
    background:          Color(0xFFF2F2F2),
    containerTop:        Color(0xFFFFFFFF),
    containerBottom:     Color(0xFFFFFFFF),
    borderColor:         Color(0xFFDEDEDE),
    appBarTop:           Color(0xFFFFFFFF),
    appBarBottom:        Color(0xFFFFFFFF),
    appBarBorder:        Color(0xFFDCDCDC),
    searchBarBackground: Color(0xFFFFFFFF),
    searchBarBorder:     Color(0xFFE0E0E0),
  );

  // ── 3. DARK COLOR SET ───────────────────────────────────────────
  // Replace these values with your actual dark-mode palette.
  static const AppColorsExtension dark = AppColorsExtension(
    foreground:          Color(0xFFC0CAF5),
    background:          Color(0xFF1E2030),
    containerTop:        Color(0xFF3D4070),
    containerBottom:     Color(0xFF2A2E50),
    borderColor:         Color(0xFF4A5580),
    appBarTop:           Color(0xFF252840),
    appBarBottom:        Color(0xFF111320),
    appBarBorder:        Color(0xFF383950),
    searchBarBackground: Color(0xFF252840),
    searchBarBorder:     Color(0xFF44456A),
  );

  // ── 4. copyWith ─────────────────────────────────────────────────
  // Required by ThemeExtension. Enables partial color overrides.
  // Every field here must mirror the field declarations above.
  @override
  AppColorsExtension copyWith({
    Color? foreground,
    Color? background,
    Color? containerTop,
    Color? containerBottom,
    Color? borderColor,
    Color? appBarTop,
    Color? appBarBottom,
    Color? appBarBorder,
    Color? searchBarBackground,
    Color? searchBarBorder,
  }) {
    return AppColorsExtension(
      foreground:          foreground          ?? this.foreground,
      background:          background          ?? this.background,
```

```
      containerTop:        containerTop        ?? this.containerTop,
      containerBottom:     containerBottom     ?? this.containerBottom,
      borderColor:         borderColor         ?? this.borderColor,
      appBarTop:           appBarTop           ?? this.appBarTop,
      appBarBottom:        appBarBottom        ?? this.appBarBottom,
      appBarBorder:        appBarBorder        ?? this.appBarBorder,
      searchBarBackground: searchBarBackground ?? this.searchBarBackground,
      searchBarBorder:     searchBarBorder     ?? this.searchBarBorder,
    );
  }

  // ── 5. lerp ──────────────────────────────────────────────────────────
  // Required by ThemeExtension. Powers smooth animated theme transitions.
  // Color.lerp interpolates between two colors at position t (0.0 → 1.0).
  @override
  AppColorsExtension lerp(
    ThemeExtension<AppColorsExtension>? other,
    double t,
  ) {
    if (other is! AppColorsExtension) return this;
    return AppColorsExtension(
      foreground: Color.lerp(foreground, other.foreground, t)!,
      background: Color.lerp(background, other.background, t)!,
      containerTop: Color.lerp(containerTop, other.containerTop, t)!,
      containerBottom: Color.lerp(containerBottom, other.containerBottom, t)!,
      borderColor: Color.lerp(borderColor, other.borderColor, t)!,
      appBarTop: Color.lerp(appBarTop, other.appBarTop, t)!,
      appBarBottom:Color.lerp(appBarBottom, other.appBarBottom, t)!,
      appBarBorder: Color.lerp(appBarBorder, other.appBarBorder, t)!,
      searchBarBackground: Color.lerp(searchBarBackground,
other.searchBarBackground, t)!,
      searchBarBorder: Color.lerp(searchBarBorder, other.searchBarBorder, t)!,
    );
  }
}
```

📌 **NOTE:** copyWith and lerp are mandatory abstract methods from ThemeExtension — the compiler will not build without both. When you add a new field, the compiler immediately flags every place you forgot to update: the constructor, copyWith, lerp, and both static color sets.

Step 2	app_styles.dart — The Unified Style Interface

AppStyles is the only file any widget should ever import for colors, text styles, and dimensions. Widgets never import AppColorsExtension directly — they always go through AppStyles. This keeps widget code clean and makes future refactoring trivial.

Color Category	When to Use
`AppStyles.colors(context).field`	Colors that differ between Light and Dark — text, backgrounds, borders, containers
`AppStyles.staticColor`	Colors that are identical in both themes — brand/accent colors like your primary blue or error red
`AppStyles.dynamicGetter(context)`	Pre-composed colors with a fixed opacity or transformation applied to a dynamic base color

The Bridge Helper — colors(context)

The most important method in the file. It retrieves the active AppColorsExtension from the theme tree. All dynamic color access flows through it.

```
static AppColorsExtension colors(BuildContext context) {
  return Theme.of(context).extension<AppColorsExtension>()!;
}

// Usage in any widget:
color: AppStyles.colors(context).foreground
color: AppStyles.colors(context).background
color: AppStyles.colors(context).borderColor
```

Static Colors — Theme-Invariant Brand Colors

These are brand and accent colors that look correct on both light and dark backgrounds. They are declared as static const — no context required.

```
// Declare your brand palette here.
// Access anywhere without context: AppStyles.primary, AppStyles.error, etc.

static const Color primary = Color(0xFF4A90E2); // your primary brand color
static const Color secondary = Color(0xFF7C5CBF); // your secondary brand color
```

```
static const Color success = Color(0xFF4CAF82); // positive states
static const Color warning = Color(0xFFFF9F43); // caution states
static const Color error = Color(0xFFE85D6A); // error states
static const Color info = Color(0xFF6ADFFF); // informational
static const Color white = Color(0xFFFFFFFF);
static const Color black = Color(0xFF000000);
static const Color transparent = Colors.transparent;
static Color get shadow => Colors.black54;

// Add any additional brand-specific static colors below.
```

Dynamic Getters — Pre-composed Color Utilities

Dynamic getters apply a consistent transformation to a base dynamic color. Defining them once here means every gradient, border, and overlay across the app uses exactly the same opacity — and if you ever want to adjust it globally, one line changes.

```
// Card / container gradient stops
static Color gradientTop(BuildContext context) =>
    colors(context).containerTop.withValues(alpha: 0.80);

static Color gradientBottom(BuildContext context) =>
    colors(context).containerBottom.withValues(alpha: 0.65);

// App bar gradient stops
static Color appBarGradientTop(BuildContext context) =>
    colors(context).appBarTop.withValues(alpha: 0.80);

static Color appBarGradientBottom(BuildContext context) =>
    colors(context).appBarBottom.withValues(alpha: 0.65);

// Standard border with consistent opacity
static Color border(BuildContext context) =>
    colors(context).borderColor.withValues(alpha: 0.75);

// Usage:
// gradient: LinearGradient(colors: [
//   AppStyles.gradientTop(context),
//   AppStyles.gradientBottom(context),
// ])
// border: Border.all(color: AppStyles.border(context))
```

Text Style Helpers

All text styles are static methods that require BuildContext. This ensures the correct foreground color is resolved automatically for the active theme. Always use these helpers for consistency — never hard-code a color in a TextStyle.

```
// ── Heading / title ──────────────────────────────────────────────
// Basic:    AppStyles.title(context)
// Override: AppStyles.title(context, fontSize: 28, color: AppStyles.primary)
static TextStyle title(
  BuildContext context, {
  double fontSize = 22,
  Color? color,
}) => TextStyle(
  color:      color ?? colors(context).foreground,
  fontSize:   fontSize,
  fontWeight: FontWeight.w800,
  height:     1.2,
);

// ── Semibold body text ───────────────────────────────────────────
// Basic:    AppStyles.labelMedium(context)
// Override: AppStyles.labelMedium(context, color: AppStyles.primary, fontSize:
14)
static TextStyle labelMedium(
  BuildContext context, {
  double fontSize = 15,
  Color? color,
}) => TextStyle(
  color:      color ?? colors(context).foreground,
  fontSize:   fontSize,
  fontWeight: FontWeight.w600,
);

// ── Bold body text ───────────────────────────────────────────────
static TextStyle labelBold(
  BuildContext context, {
  double fontSize = 15,
  Color? color,
}) => TextStyle(
  color:      color ?? colors(context).foreground,
  fontSize:   fontSize,
  fontWeight: FontWeight.w800,
);

// Add additional text style helpers following the same pattern.
```

Dimensions & Spacing Constants

Centralise all shared layout values here so every screen uses consistent sizing. These are pure constants — no context required.

```
static const double borderWidth   = 2.0;
static const double cornerRadius  = 16.0;
static const double fieldRadius   = 12.0;
static const double dialogRadius  = 24.0;

static const EdgeInsets screenPadding = EdgeInsets.symmetric(
  horizontal: 20, vertical: 16,
);

// Responsive sizing helpers
static double responsiveWidth(BuildContext context, double fraction) {
  final w = MediaQuery.sizeOf(context).width * fraction;
  return w > 480 ? 480 : w;
}
```

Step 3	theme_cubit.dart — Theme State Management

ThemeCubit manages exactly one value: the active ThemeMode. It handles two responsibilities — persisting the user's preference so it survives app restarts, and broadcasting the new state so every widget rebuilds instantly.

Complete File

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:shared_preferences/shared_preferences.dart';

class ThemeCubit extends Cubit<ThemeMode> {

  // The key string used to read/write the preference.
  // Must match the key used in main.dart exactly.
  static const String _themeKey = 'theme_mode';

  // Receives the initial theme resolved by main.dart.
  // Passes it to super() so the Cubit's first state is already correct.
  ThemeCubit(super.initialTheme);

  Future<void> toggleTheme(bool isCurrentlyLight) async {
    final prefs = await SharedPreferences.getInstance();

    if (isCurrentlyLight) {
      // Persist first, then update UI.
      // This order guarantees the preference is saved even if a crash follows.
      await prefs.setString(_themeKey, 'dark');
      emit(ThemeMode.dark);
    } else {
      await prefs.setString(_themeKey, 'light');
      emit(ThemeMode.light);
    }
  }
}
```

Design Decisions Explained

Why the parameter is bool isCurrentlyLight, not void toggleTheme()

The function receives the current theme as a boolean rather than reading it from state internally. The calling widget already knows the current theme (it evaluated isLightMode in its build method to decide which icon to show). Passing it in avoids an unnecessary state.state read and keeps the Cubit's interface intention-revealing — the caller is explicitly saying 'I am in light mode, switch me to dark'.

Why prefs.setString is called before emit

The disk write always happens before the UI update. If the device crashes or the app is force-quit in the nanoseconds after emit but before the write, the user would see the new theme on screen but the app would revert to the old theme on next launch. Saving first makes the toggle atomic from the user's perspective.

Why ThemeCubit extends Cubit<ThemeMode> and not Cubit<bool>

Using Flutter's built-in ThemeMode enum makes the state self-documenting and interoperates directly with MaterialApp's themeMode parameter without any conversion logic.

> 💡 **TIP:** Register ThemeCubit with registerLazySingleton in your dependency injection setup, not registerFactory. The theme must persist as a single shared instance across every screen for the entire app lifetime.

Step 4	main.dart — Startup, Wiring & App Root

main.dart performs three sequential jobs on startup: it reads the persisted theme before drawing the first frame, wires the ThemeCubit into the root of the widget tree, and connects the active ThemeMode to MaterialApp.

Complete File

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'core/theming/app_colors_extension.dart';
import 'core/theming/theme_cubit.dart';
// import your router here

void main() async {
  // Required before any async work in main().
  WidgetsFlutterBinding.ensureInitialized();

  // Read the saved preference BEFORE runApp so the first frame
  // is drawn in the correct theme — no flicker on launch.
  final prefs      = await SharedPreferences.getInstance();
  final savedTheme = prefs.getString('theme_mode');

  // Default to dark if no preference has been saved yet.
  // Change ThemeMode.dark to ThemeMode.light to default to light instead.
  final initialTheme =
      savedTheme == 'light' ? ThemeMode.light : ThemeMode.dark;

  runApp(MyApp(initialTheme: initialTheme));
}

class MyApp extends StatelessWidget {
  final ThemeMode initialTheme;
  const MyApp({ super.key, required this.initialTheme });

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      // ThemeCubit is created once at the root and available to every
      // screen, dialog, and overlay without any additional setup.
      create: (_) => ThemeCubit(initialTheme),

      child: BlocBuilder<ThemeCubit, ThemeMode>(
        builder: (context, currentMode) {
          return MaterialApp.router(
```

```
            debugShowCheckedModeBanner: false,
            title: 'Your App Name',

            // Light theme — Flutter uses this when currentMode is
ThemeMode.light.
            theme: ThemeData(
              brightness: Brightness.light,
              extensions: const <ThemeExtension<dynamic>>[
                AppColorsExtension.light,
              ],
            ),

            // Dark theme — Flutter uses this when currentMode is ThemeMode.dark.
            darkTheme: ThemeData(
              brightness: Brightness.dark,
              extensions: const <ThemeExtension<dynamic>>[
                AppColorsExtension.dark,
              ],
            ),

            // The live connection: BlocBuilder re-runs when ThemeCubit emits,
            // passing the new mode here and triggering a full theme swap.
            themeMode: currentMode,

            routerConfig: yourRouter,
          );
        },
      ),
    );
  }
}
```

Why Each Part Exists

WidgetsFlutterBinding.ensureInitialized()

This initialises the Flutter binding layer, which is required before any platform channel communication (like SharedPreferences reading from disk) can happen in an async main function. Omitting it causes a runtime crash.

Reading SharedPreferences before runApp

If the saved theme were read after runApp, Flutter would draw one frame in the default system theme, then immediately redraw in the saved theme. On most devices this produces a visible white flash at startup. Reading before runApp means the first frame is always correct.

BlocProvider at the root, outside MaterialApp

Providing ThemeCubit at the root means every part of the app — including dialogs and bottom sheets that create their own overlay entries — can call context.read<ThemeCubit>() without any additional BlocProvider setup in individual screens.

extensions: [AppColorsExtension.light] and [...dark]

This attaches your custom color sets to Flutter's theme data objects. When themeMode is ThemeMode.light, Flutter activates the theme property and AppColorsExtension.light becomes the active extension, retrieved by any widget calling Theme.of(context).extension<AppColorsExtension>(). The switch between the two sets is entirely automatic.

> ⚠️ **WARNING:** Never call SharedPreferences again in the widget tree to check the current theme. The ThemeCubit's state IS the current theme. Reading from disk bypasses the reactive state system and can return a stale value.

Using Colors in Widgets

Four patterns cover every color usage scenario you will encounter. Learn them in order — each builds on the previous.

Pattern 1 — Dynamic Color

Use for any color that must differ between light and dark mode.

```
// Any widget, any file:
Text(
  'Hello',
  style: TextStyle(color: AppStyles.colors(context).foreground),
)

Container(color: AppStyles.colors(context).background)

// With an inline opacity adjustment:
Container(
  color: AppStyles.colors(context).borderColor.withValues(alpha: 0.40),
)
```

Pattern 2 — Static Color

Use for brand colors and accents that look correct on both themes.

```
// No context needed:
ElevatedButton(
  style: ElevatedButton.styleFrom(backgroundColor: AppStyles.primary),
  child: const Text('Confirm'),
)

Icon(Icons.check_circle, color: AppStyles.success)
Icon(Icons.error, color: AppStyles.error)
```

Pattern 3 — Dynamic Getter

Use the pre-built getters for gradients and borders to guarantee consistent opacity across every screen.

```
// Card gradient:
gradient: LinearGradient(
  begin: Alignment.topCenter,
  end: Alignment.bottomCenter,
  colors: [
    AppStyles.gradientTop(context),
    AppStyles.gradientBottom(context),
  ],
)

// Border:
border: Border.all(color: AppStyles.border(context))

// App bar gradient:
colors: [
  AppStyles.appBarGradientTop(context),
  AppStyles.appBarGradientBottom(context),
]
```

Pattern 4 — Theme-Based Conditional Logic

When a widget needs to behave differently per theme — not just a different color, but a different icon, different padding, or a conditional child — use the isLightMode boolean.

```
// Always define at the TOP of build(), before any widget is returned.
final bool isLightMode =
    Theme.of(context).brightness == Brightness.light;

// ── Examples ──────────────────────────────────────────────────────

// Different color per theme:
color: isLightMode ? AppStyles.black : AppStyles.white,

// Ternary with a nullable color (wrap ternary in parentheses):
// ❌ WRONG:  overrideColor ?? isLightMode ? colorA : colorB
// ✅ CORRECT:
color: overrideColor ?? (isLightMode ? colorA : colorB),

// Different icon per theme:
icon: isLightMode ? Icons.nightlight_round : Icons.wb_sunny_rounded,
```

Triggering a Theme Toggle

Call toggleTheme from any widget. The most common placement is a button in the app bar or the login screen.

```
// Full toggle button example:
@override
Widget build(BuildContext context) {
  final bool isLightMode =
      Theme.of(context).brightness == Brightness.light;

  return IconButton(
    onPressed: () {
      // context.read — sends a command, does not listen.
      // Use read for actions. Use watch only when the widget must rebuild
      // every time the state changes.
      context.read<ThemeCubit>().toggleTheme(isLightMode);
    },
    icon: Icon(
      isLightMode ? Icons.nightlight_round : Icons.wb_sunny_rounded,
      color: isLightMode ? AppStyles.primary : AppStyles.warning,
    ),
  );
}
```

Common Pitfalls

Every pitfall below has a specific cause. Understanding the cause is more valuable than memorising the fix.

Pitfall 1 — The const Conflict

Dynamic colors are resolved at runtime via context. Any widget that contains them cannot be const, which is a compile-time construct. The compiler will flag this immediately.

```
// ❌ WRONG – compile error:
const Text(
  'Hello',
  style: TextStyle(color: AppStyles.colors(context).foreground),
)

// ✅ CORRECT – remove const from the widget:
Text(
  'Hello',
  style: TextStyle(color: AppStyles.colors(context).foreground),
)

// The fix also applies to every ancestor widget:
// ❌ The Column wrapping this Text cannot be const either.
// Remove const from the entire branch that contains a dynamic color.
// You CAN keep const on sibling branches that contain no dynamic colors.
```

> ⚠️ **WARNING:** If the error persists after removing const from the widget itself, walk up the tree. A parent Column, Padding, or SizedBox may also be marked const and needs the same fix.

Pitfall 2 — Context-Based Default Constructor Parameters

Dart requires default parameter values to be compile-time constants. You cannot call colors(context) — a runtime operation — as a default value inside a constructor.

```
// ❌ WRONG – compile error:
class CardWidget extends StatefulWidget {
  final Color fillColor;
  const CardWidget({
    this.fillColor = AppStyles.colors(context).background, // ILLEGAL
  });
```

```
}

// ✅ CORRECT – make the parameter nullable and resolve inside build():
class CardWidget extends StatefulWidget {
  final Color? fillColor; // nullable – no default at declaration
  const CardWidget({ this.fillColor });
}

// Inside State<CardWidget>.build():
@override
Widget build(BuildContext context) {
  // Resolve the actual color here, where context is available.
  final Color effectiveFill =
      widget.fillColor ?? AppStyles.colors(context).background;

  return Container(color: effectiveFill);
}
```

Pitfall 3 — Operator Precedence: ?? vs ? :

Dart gives the null-coalescing operator (??) higher precedence than the ternary operator (? :). Mixing them without parentheses produces a type error because Dart evaluates the wrong part first.

```
// ❌ WRONG – Dart parses as: (overrideColor ?? isLightMode) ? A : B
// This is a type error: (Color? ?? bool) is not a valid bool.
color: overrideColor ?? isLightMode ? colorA : colorB,

// ✅ CORRECT – parentheses make the intended grouping explicit:
color: overrideColor ?? (isLightMode ? colorA : colorB),
```

Pitfall 4 — isLightMode Inside Dialog Builders

When a dialog is opened, Flutter creates a new overlay entry with its own BuildContext. If you captured isLightMode from the screen's context before the dialog opened, it is technically the overlay context and can become stale. Always re-evaluate isLightMode inside the dialog's own builder.

```
// ❌ WRONG – isLightMode from the screen's context, not the dialog's:
final bool isLightMode = Theme.of(context).brightness == Brightness.light;
showGeneralDialog(
  context: context,
```

```
  pageBuilder: (_, __, ___) => StatefulBuilder(
    builder: (dialogContext, setDialogState) {
      // isLightMode here is from the outer scope — potentially stale.
      return Container(color: isLightMode ? Colors.white : Colors.black);
    },
  ),
);

// ✅ CORRECT — evaluate brightness inside the dialog's own builder:
showGeneralDialog(
  context: context,
  pageBuilder: (_, __, ___) => StatefulBuilder(
    builder: (dialogContext, setDialogState) {
      // Re-evaluated using dialogContext — always fresh and correct.
      final bool isLightMode =
          Theme.of(dialogContext).brightness == Brightness.light;

      return Container(color: isLightMode ? Colors.white : Colors.black);
    },
  ),
);
```

How to Add a New Color

Every time your design requires a new color, use this decision tree to find the right place for it before writing any code.

```
Does this color need to be DIFFERENT between Light and Dark mode?
│
├─ YES ──▶ Add it to AppColorsExtension
│         Follow these 5 steps in order:
│         1. Add the field:  final Color yourField;
│         2. Add it to the constructor:  required this.yourField,
│         3. Add a light value inside the 'light' static const
│         4. Add a dark value inside the 'dark' static const
│         5. Add it to copyWith (nullable param + ?? fallback)
│         6. Add it to lerp: Color.lerp(yourField, other.yourField, t)!,
│         Access: AppStyles.colors(context).yourField
│
└─ NO ───▶ Is the same value used in many places across the app?
          │
          ├─ YES, with a fixed opacity transformation ──▶
          │   Add a dynamic getter to AppStyles
          │   static Color myUtility(BuildContext context) =>
          │     colors(context).someBase.withValues(alpha: 0.55);
          │   Access: AppStyles.myUtility(context)
          │
          └─ NO, same in both themes with no transformation ──▶
             Add a static const to AppStyles
             static const Color myBrandColor = Color(0xFFXXXXXX);
             Access: AppStyles.myBrandColor
```

> 💡 **TIP:** When you add a new field to AppColorsExtension, let the Dart compiler guide you. It will immediately highlight the exact lines in copyWith and lerp that need updating. Follow the red underlines as a checklist — you cannot miss anything.

Quick Reference Cheat Sheet

I need...	Write this	Defined in
Text / icon color (theme-aware)	`AppStyles.colors(context).foreground`	AppColorsExtension
Screen background	`AppStyles.colors(context).background`	AppColorsExtension
Card gradient top	`AppStyles.gradientTop(context)`	AppStyles getter

I need...	Write this	Defined in
Card gradient bottom	AppStyles.gradientBottom(context)	AppStyles getter
App bar gradient top	AppStyles.appBarGradientTop(context)	AppStyles getter
Standard border	AppStyles.border(context)	AppStyles getter
Primary brand color	AppStyles.primary	AppStyles static
Success / error color	AppStyles.success / AppStyles.error	AppStyles static
Title text style	AppStyles.title(context)	AppStyles method
Semibold text style	AppStyles.labelMedium(context)	AppStyles method
Is the app in light mode?	Theme.of(context).brightness == Brightness.light	Flutter API
Toggle the theme	context.read<ThemeCubit>().toggleTheme(isLightMode)	ThemeCubit
Watch theme (rebuild on change)	context.watch<ThemeCubit>().state	ThemeCubit

Golden Rules

- **One import per widget.** Widgets import app_styles.dart only. Never import app_colors_extension.dart directly in a widget file.
- **Name colors by role, never by value.** foreground not darkSlateBlue. background not offWhite.
- **Remove const from dynamic-color widgets.** Any widget — or its ancestors — that contains AppStyles.colors(context) cannot be marked const.
- **Nullable parameters, not context-based defaults.** Use Color? in constructors and resolve the actual color inside build() using ??.
- **Parenthesise ternaries after ??.** Always write: color ?? (isLightMode ? A : B), never: color ?? isLightMode ? A : B.
- **Re-evaluate isLightMode inside dialog builders.** Always define it from the dialog's own BuildContext, not the screen's.
- **Persist before emitting.** In ThemeCubit, always call prefs.setString before emit so the preference is saved even if the app crashes immediately after.
- **Never re-read SharedPreferences for the theme.** The ThemeCubit's state is the live source of truth. Reading from disk gives you a stale snapshot.

- **Use context.read for commands, context.watch for listening.** A toggle button fires a command — use read. A widget that rebuilds when the theme changes — use watch or BlocBuilder.
- **Let the compiler guide new field additions.** Adding a field to AppColorsExtension will produce compile errors at every location that needs updating. Follow them as your checklist.